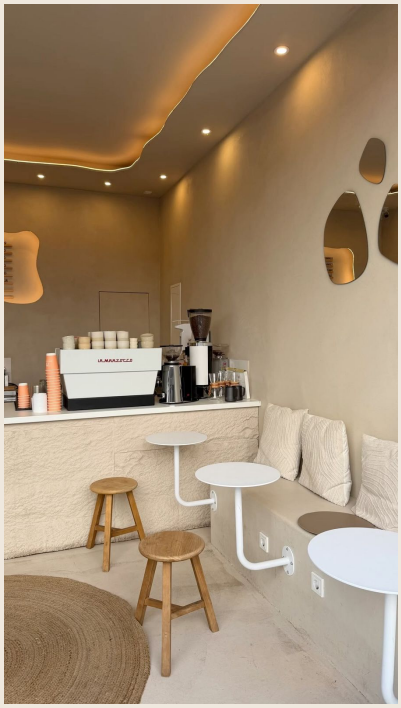
Department of Computer Science and Information Technology
CSCI426: Advanced Web Programming · Project Phase 1

Bean & Brew

A coffee shop ordering website built with React

Theme 4: Restaurant Ordering System (menu display, cart and order tracking)



Student(s)	[Student name] — [Student ID] [Student name] — [Student ID]
Instructor	[Instructor name]
GitHub repository	[https://github.com/your-username/bean-and-brew]
Live website	[https://your-project.vercel.app]
Semester	Fall 2026

Abstract

Customers of busy coffee shops lose time standing in line just to read the menu and place an order, and they have no way of knowing when their drink will be ready. **Bean & Brew** is a responsive frontend web application that moves this process online. Customers browse a real café menu of more than 120 items through tabs for four categories, choose drink sizes, build a cart, place a takeaway or dine-in order, and then follow the order through four live stages: received, preparing, ready and completed.

The application is built with React 18 and React Router 6 and has seven routes, including two dynamic pages (*/menu/:categoryId* and */order/:orderId*). The shopping cart is shared between pages with the Context API, and the cart, orders and theme are kept in the browser with localStorage. The interface uses a custom CSS design system inspired by the café interior, supports light and dark mode, and adapts from wide desktop screens down to 360 px phones. The source code is managed with Git and published on GitHub, and the site is deployed on Vercel.

Introduction

The project follows Theme 4 of the assignment, the Restaurant Ordering System, applied to a specialty coffee shop. Its goals are:

- Display a complete, easy to scan menu with prices per size.
- Let customers order without an account in under a minute.
- Show the status of an order so customers know when to collect it.
- Present the café story and contact details on separate pages.
- Work equally well on a laptop and on a phone.

System Design

1. Architecture

The application is a single-page React app. *main.jsx* wraps everything in a *HashRouter* and a *CartProvider*. *App.jsx* renders the shared layout (navbar and footer) and decides which page component to show for the current URL. Pages read static menu data from *data/menu.js*, share the cart through *CartContext*, and save orders through helper functions in *utils/orders.js*.

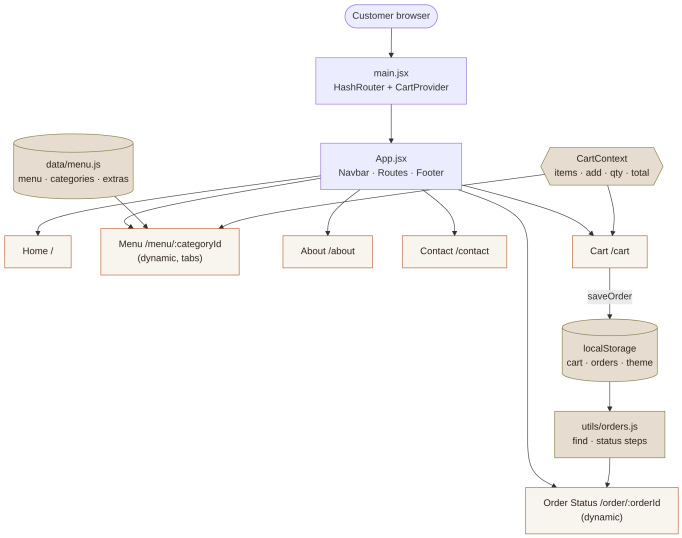


Figure 1. Component and data architecture

2. Pages and routes

Route	Page	Purpose
/	Home	Hero with café photo, opening facts and house favourites
/menu/:categoryId	Menu (dynamic)	Tabs at the top switch between drinks, food, bars and sweets; one component renders each based on the URL. /menu opens drinks.
/about	About	Short café story
/contact	Contact	Address, phone, hours and a validated message form
/cart	Cart	Edit quantities, enter details, choose takeaway or dine-in, place order
/order/:orderId	Order Status (dynamic)	Shown after checkout: live timeline and receipt for one order
*	Not Found	Friendly 404 page

3. User flow



Figure 2. Ordering flow from menu to tracking

4. Data model

All data lives in plain JavaScript objects, so the app needs no backend for Phase 1:

Object	Fields	Where
Menu section	id, title, sized, items[], note	data/menu.js
Menu item	name, prices [S, M, L] or price, tag, desc	data/menu.js
Category	id, label, icon, blurb, sections[]	data/menu.js
Cart item	key, name, size, price, qty	CartContext (state + localStorage)
Order	id, name, phone, type, table, notes, items[], total, createdAt	localStorage "bb-orders"

5. State management

- **Local state** (*useState*) for form fields, validation errors, search text and the mobile menu.
- **Global state** (*useContext*) for the cart, because the navbar badge, category pages and cart page all need it.
- **Persistent state** (*localStorage*) so the cart, orders and theme survive a page refresh.
- **Derived state**: cart count, total and the order's current step are calculated, not stored.

6. UI/UX and responsive design

- A colour palette taken from the café photo: sand walls, terracotta cups and olive accents, stored as CSS custom properties.
- Dark mode redefines the same variables, so every component switches theme without extra rules.
- Typography pairs Playfair Display for headings with Poppins for body text.
- Drink prices are buttons laid out in Small / Medium / Large columns; one tap adds the item and shows a .
- CSS Grid and Flexbox with breakpoints at 1000, 880, 860 and 700 px: the menu goes from two columns to one, grids stack, and the navbar becomes a hamburger menu.
- Accessibility: labelled form fields, aria-labels on icon buttons, visible focus outlines and reduced-motion support.

Technologies Used

Technology	Use in the project
React 18	Component-based UI with hooks (useState, useEffect, useContext)
React Router 6	Client-side routes, two dynamic routes, NavLink active states, useNavigate, Navigate
Context API	Shared shopping cart across pages
Vite 5	Fast development server and production build
HTML5 & CSS3	Semantic markup; custom properties, Grid, Flexbox, media queries
localStorage	Saves cart, orders and theme in the browser
Git & GitHub	Version control and source code hosting
Vercel / GitHub Pages	Hosting the deployed website
Google Fonts	Playfair Display and Poppins typefaces

Code Snippets

Routing with a dynamic route (App.jsx)

```
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/menu" element={<Navigate to="/menu/drinks" replace />} />
          <Route path="/menu/:categoryId" element={<Category />} />
          <Route path="/about" element={<About />} />
          <Route path="/contact" element={<Contact />} />
          <Route path="/cart" element={<Cart />} />
          <Route path="/order/:orderId" element={<OrderStatus />} />
          <Route path="*" element={<NotFound />} />
        </Routes>
```

Dynamic category page reading the URL (Category.jsx)

```
export default function Category() {
  const { categoryId } = useParams()
  const category = getCategory(categoryId)
  const { count, total } = useCart()

  if (!category) {
  ...
```

Adding to the shared cart (CartContext.jsx)

```
  // size is 'S', 'M', 'L' or null for single-price items
  const addToCart = (name, size, price) => {
    const key = size ? `${name}-${size}` : name
    setItems((prev) => {
      const found = prev.find((i) => i.key === key)
      if (found) return prev.map((i) => (i.key === key ? { ...i, qty: i.qty + 1 } : i))
      return [...prev, { key, name, size, price, qty: 1 }]
    })
```

Order tracking logic (utils/orders.js)

```
// Each step starts a number of seconds after the order is placed (sped up for the demo)
export const STEPS = [
  { label: 'Order received', after: 0 },
  { label: 'Preparing', after: 30 },
  { label: 'Ready', after: 90 },
  { label: 'Completed', after: 180 },
]

export function currentStep(order, now = Date.now()) {
  const seconds = (now - order.createdAt) / 1000
  let step = 0
  STEPS.forEach((s, i) => {
    if (seconds >= s.after) step = i
  })
  return step
}
```

Live refresh on the order page (OrderStatus.jsx)

```
  // Re-check the status every 5 seconds
  useEffect(() => {
    const timer = setInterval(() => setNow(Date.now()), 5000)
    return () => clearInterval(timer)
  }, [])
```

Theme variables and dark mode (index.css)

```
:root {
  --bg: #efe8de;
  --surface: #f8f4ee;
  --sunken: #e6ddd0;
  --text: #2e241c;
  --muted: #76665a;
  --primary: #b9582a;   /* terracotta */
  ...
}

body.dark {
  --bg: #1d1814;
  --surface: #28211b;
  --primary: #e0874f;
  ...
}
```

User Interface

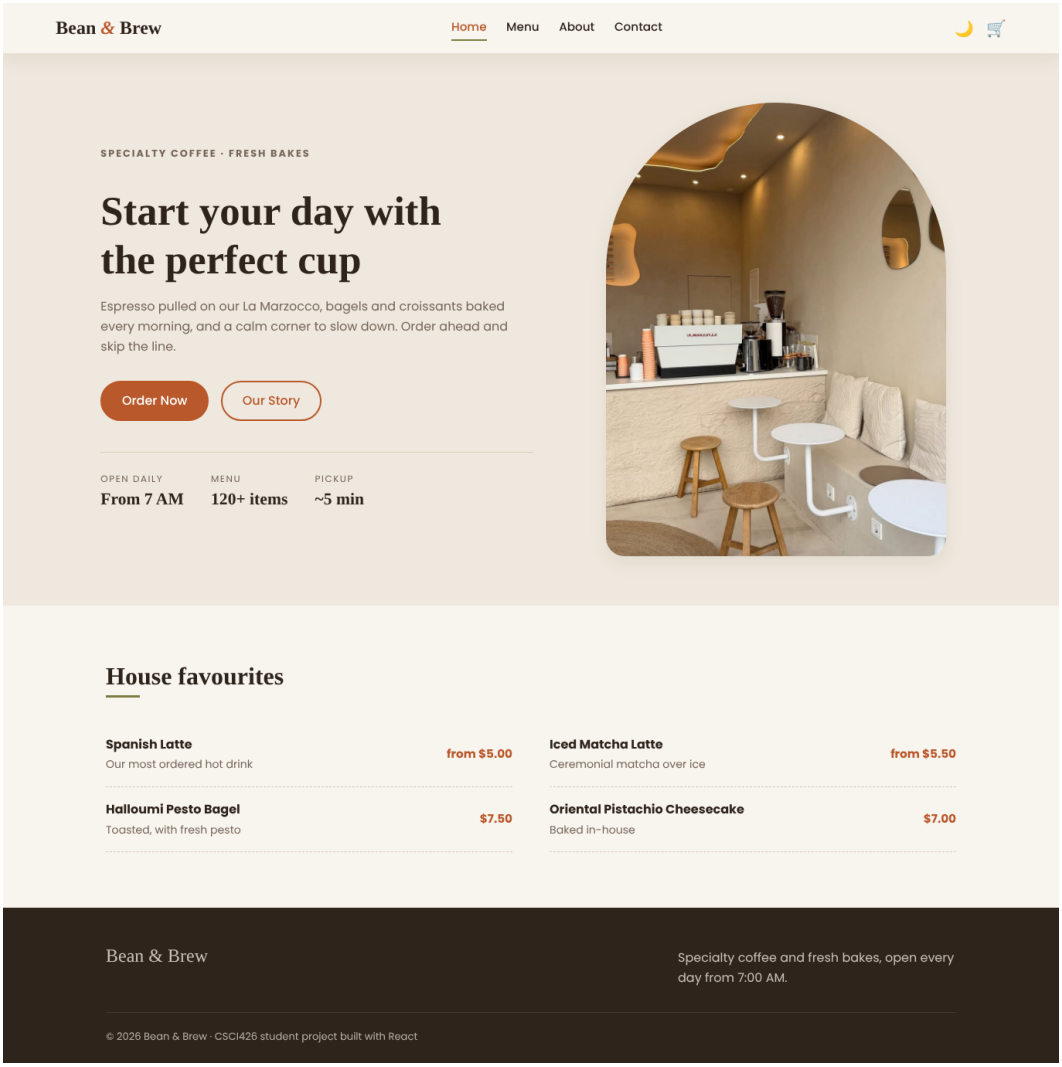


Figure 3. Home page

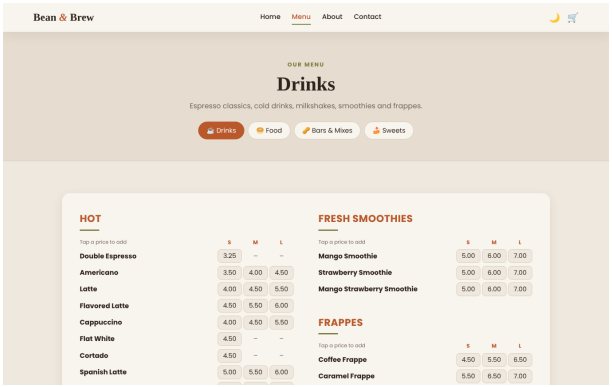


Figure 4. Menu page with category tabs

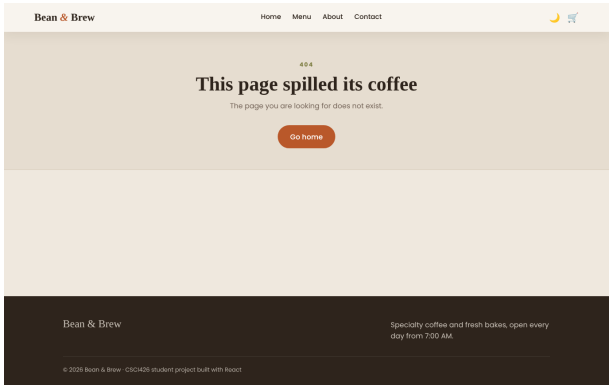


Figure 5. 404 page

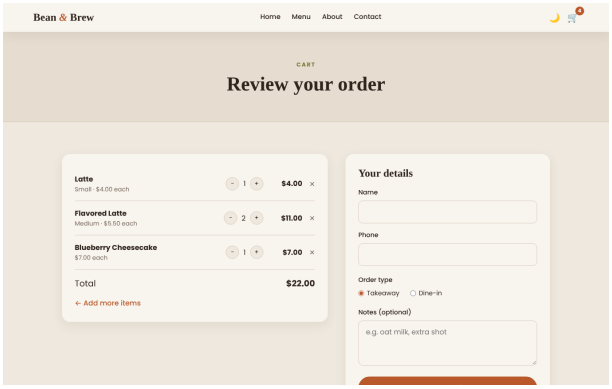


Figure 6. Cart and checkout

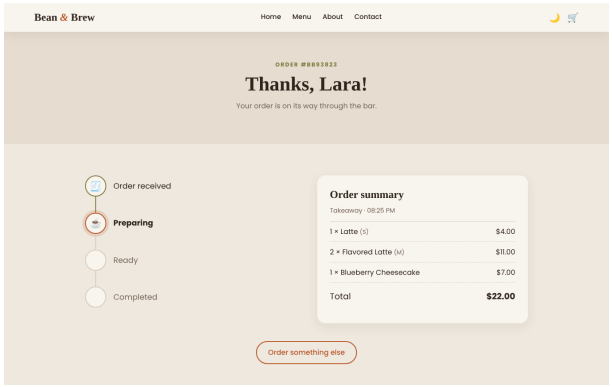


Figure 7. Live order tracking

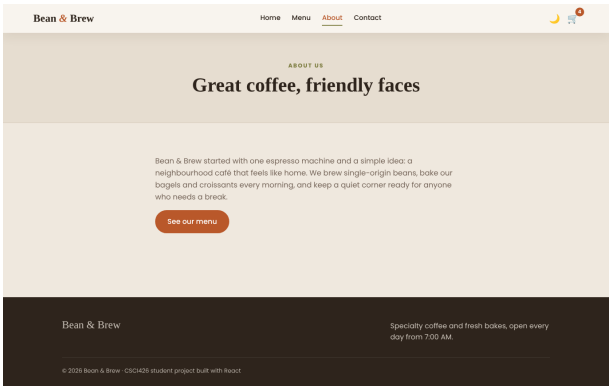


Figure 8. About

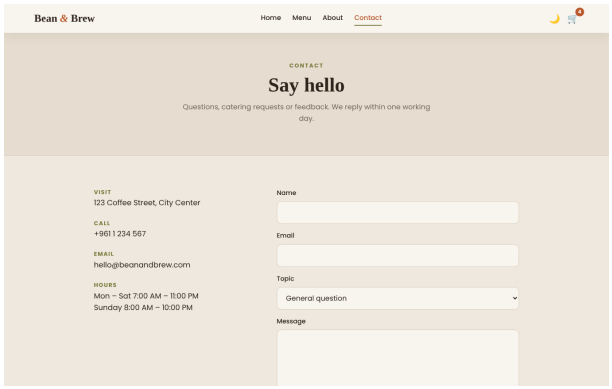


Figure 9. Contact form

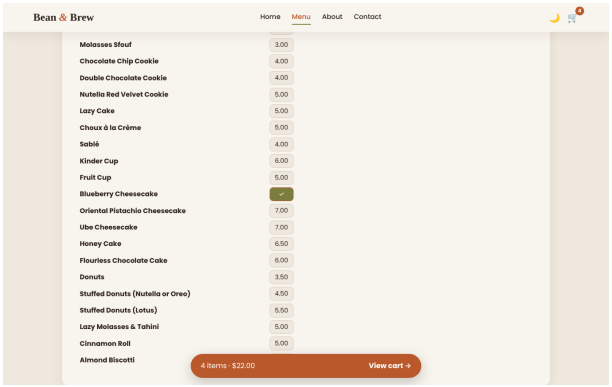


Figure 10. Sweets tab

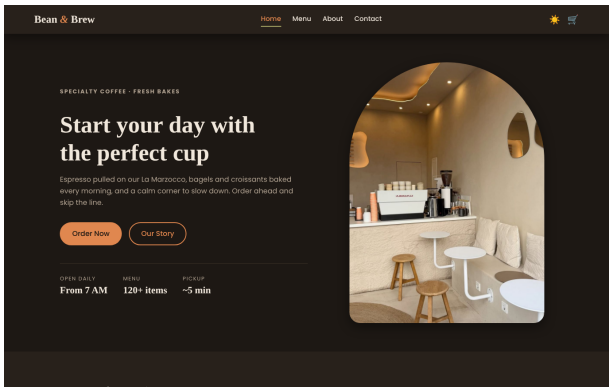


Figure 11. Dark mode

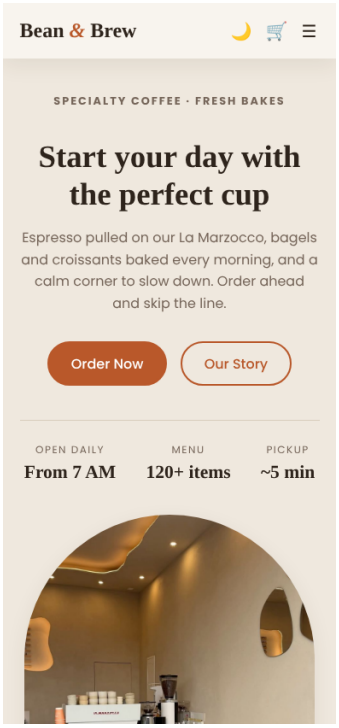


Figure 12. Mobile home

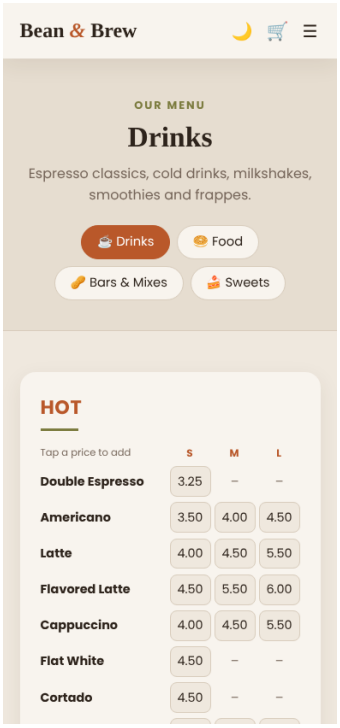


Figure 13. Mobile menu

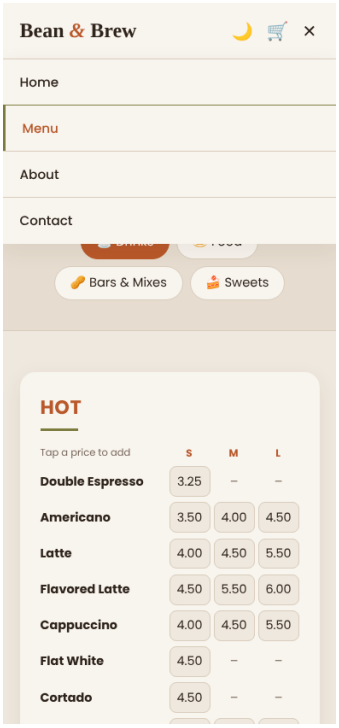


Figure 14. Mobile navigation

Version Control and Deployment

The project was developed in small steps, each saved as a Git commit and pushed to GitHub. The commit history below shows how the application was built feature by feature:

Commit	Message
f7b777f	Set up React + Vite project
4cfe1d3	Add full cafe menu data with sizes, categories and add-ons
59aafd6	Add base styles, colour theme and dark mode
e29af45	Add layout: navbar, footer and page header components
152eb96	Add home page with cafe photo and menu categories
1b84e51	Add menu page with search and dynamic category pages
7df557e	Add services, about, contact and 404 pages
b4551b6	Add shopping cart with Context API and checkout form
569a859	Add order saving and live order tracking pages
a6ecf7f	Wire up routes in App
0b6e0f0	Add README with setup, deployment and screenshots
1c86e39	Add project report (PDF)
b508b38	Simplify site: menu tabs, shorter home and about, remove services and track pages
4ebaa36	Update README and screenshots for new page layout
495ea95	Update project report
62f21d7	Remove leftover code from deleted pages and improve phone layout
f103b86	Refresh screenshots

Deployment

- The repository is imported into Vercel, which detects Vite and runs *npm run build* on every push.
- Alternatively, *npm run deploy* publishes the *dist* folder to GitHub Pages using the gh-pages package.
- HashRouter and *base: './'* in vite.config.js make every page load correctly on both hosts.

Conclusion and Future Work

Bean & Brew meets the Phase 1 requirements: a React frontend with more than five pages including dynamic routes, a responsive design for desktop and mobile, a working cart and order tracking, Git version control, documentation and online deployment. In Phase 2 the project can be extended with a backend and database so that orders are shared between customers and baristas, a staff dashboard to update order status, user accounts with order

history, and online payment.

References and Credits

- Menu items and prices adapted from the public menu of Latte Art café, Lebanon (latteartlb.com).
- Home page photo: café interior reference photo [add source].
- React, React Router and Vite documentation (react.dev, reactrouter.com, vitejs.dev). No website template was used.

Group Contribution Statement

[Fill in. If you worked alone, keep one row and write "Individual project".]

Student name	Student ID	Contribution	%
[Name]	[ID]	[e.g. Menu data, menu and category pages, cart and checkout, styling]	[50%]
[Name]	[ID]	[e.g. Home, about, contact pages, order status page, README, report, deployment]	[50%]

Signatures: _____ _____